جامعة الأخوين

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⴰⵅⴰⵡⴰⵢⵏ

AL AKHAWAYN UNIVERSITY

School of Science and Engineering

Milestone 4: Model Development and Evaluation

Realized by:

Ayoub AKARKOUZ < 78987 >

Under the supervision of:

Pr. Asmaa MOURHIR

~ Spring 2026 ~

Table of Contents:

1. Overview: 3
2. Project Context & Integration with Previous Milestones 3
3. Project Structure and Main Files 4
4. Milestone 4 - ML Pipeline Development - Model Training and Offline Evaluation 6
   4.1 Project Structure Definition and Modularity 6
   4.2 Code Versioning 6
   4.3 Experiment Tracking and Model Versioning 7
   4.4 Integration of Model Training and Offline Evaluation into the ML Pipeline / MLOps Platform 9
   4.5 Energy Efficiency Measurement 11

1. Overview:

Milestone 4 introduces the model development and offline evaluation layer of the project. While the previous milestones established the business framing, solver logic, and data/MLOps foundation, this milestone adds a supervised machine learning workflow built on top of validated UFLP benchmark data and deterministic optimization outputs.

The implemented task is a facility-level binary classification problem. Each training example represents one (instance_id, facility_id) pair. The input is a vector of engineered numeric features derived from Milestone 3, and the target is a binary label is_open, indicating whether the facility is open in the deterministic optimal UFLP solution computed from the Milestone 2 baseline solver.

This milestone does not replace the symbolic optimization system described in Milestone 1. Instead, it adds a complementary ML pipeline for offline experimentation, facility ranking, model comparison, and future hybrid optimization support.

Figure 1: End-to-end architecture of the project after Milestone 4 integration.

2. Project Context & Integration with Previous Milestones

The project now consists of four connected layers:

- Milestone 1 defined the business problem, research framing, and the LLM-as-modeler architecture.
- Milestone 2 implemented the deterministic OR-Tools solver baseline and the LLM-generated verified solver pipeline.

- Milestone 3 implemented the data engineering and MLOps foundation, including ingestion, preprocessing, validation, feature engineering, DVC, and Feast.
- Milestone 4 implements supervised model training, offline evaluation, experiment tracking, model versioning, visualization, and ZenML pipeline orchestration.

Milestone 4 depends directly on the outputs of the previous milestones:

- Raw OR-Library UFLP benchmark files remain the source dataset.
- Milestone 3 provides engineered facility-level and instance-level features.
- Milestone 2 provides deterministic optimal facility-opening decisions used as labels.
- Milestone 4 merges those assets into a supervised dataset, trains and compares models, logs runs in MLflow, orchestrates the workflow with ZenML, and tracks optional energy usage through CodeCarbon.

3. Project Structure and Main Files

Milestone 4 is organized as a modular ML project.
The Main top-level folders:

- ***configs:*** YAML configuration files for training and model comparison.
- ***src/m4_model_dev:*** reusable source code.
- ***scripts:*** command-line entrypoints for training, comparison, tests, and ZenML runs.
- ***tests:*** regression checks.
- ***data:*** generated labels, merged datasets, and split definitions.
- ***artifacts:*** serialized trained models.
- ***reports:*** summaries, metrics, comparisons, confusion reports, figure outputs, ZenML status, and energy reports.
- ***mlruns*** and ***mlflow.db:*** MLflow local tracking store and artifacts.
- ***.zen:*** ZenML local configuration.

Explanation of the most important code modules:

- ***src/m4_model_dev/paths.py:*** centralizes project paths and runtime directories.
- ***src/m4_model_dev/data/build_labels.py:*** generates facility-open labels from the deterministic baseline solver.
- ***src/m4_model_dev/data/prepare_dataset.py:*** merges Milestone 3 features with Milestone 4 labels.
- ***src/m4_model_dev/data/make_splits.py:*** builds grouped train/validation/test splits by instance.
- ***src/m4_model_dev/models/logistic_numpy.py:*** custom NumPy logistic regression baseline.
- ***src/m4_model_dev/models/model_registry.py:*** model factory, training, scoring, serialization, and feature importance extraction.
- ***src/m4_model_dev/evaluation/metrics.py:*** computes classification metrics and confusion counts.
- ***src/m4_model_dev/reporting/training_reports.py:*** writes summaries, manifests, metrics, and training reports.
- ***src/m4_model_dev/reporting/comparison_reports.py:*** writes comparison summaries and selected-model outputs.

- ***src/m4_model_dev/reporting/figures.py***: generates PNG visualizations for training and comparison results.
- ***src/m4_model_dev/tracking/mlflow_utils.py:*** logs runs, metrics, artifacts, and registered models to MLflow.
- ***src/m4_model_dev/tracking/codecarbon_utils.py:*** activates CodeCarbon tracking when configured.
- ***src/m4_model_dev/pipelines/training_pipeline.py:*** full local training and evaluation workflow.
- ***src/m4_model_dev/pipelines/comparison_pipeline.py:*** multi-model comparison workflow.
- ***src/m4_model_dev/pipelines/zenml_pipeline.py:*** ZenML pipeline definition with visible step orchestration.

Main runnable scripts:

- ***scripts/run_train.py***
- ***scripts/run_compare.py***
- ***scripts/run_tests.py***
- ***scripts/run_zenml.py***

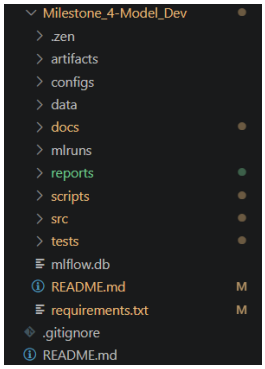


Figure 2: Repository structure of Milestone 4 showing modular separation of configs, source, reports, scripts, and artifacts.

4. Milestone 4 - ML Pipeline Development - Model Training and Offline Evaluation

4.1 Project Structure Definition and Modularity

The milestone follows a modular structure aligned with the principles of **Cookiecutter-style** data science projects and lifecycle-oriented ML engineering. The implementation avoids a monolithic script and instead separates responsibilities across distinct modules:

- data preparation
- model definition
- evaluation
- reporting
- tracking
- orchestration
- execution scripts
- tests

This modularity improves readability, reuse, maintainability, and experimentation. For example:

- label generation is isolated in ***build_labels.py***
- dataset construction is isolated in ***prepare_dataset.py***
- split logic is isolated in ***make_splits.py***
- training orchestration is isolated in ***training_pipeline.py***
- visual output generation is isolated in ***figures.py***
- experiment logging is isolated in ***mlflow_utils.py***

This structure directly supports the milestone requirement of project structure definition and modularity.

From an execution standpoint, Milestone 4 can be understood as a pipeline of reusable stages:

1. generate facility labels from the deterministic solver
2. merge labels with engineered features
3. create grouped splits
4. train the configured model
5. evaluate offline on train, validation, and test
6. write reports and PNG visualizations
7. log experiments to MLflow
8. optionally orchestrate all stages through ZenML

4.2 Code Versioning

Milestone 4 adopts a **Git-based** workflow compatible with GitHub Flow. The versioning process is documented in ***docs/versioning_and_delivery.md*** and is designed around short-lived feature branches, local validation before merge, and controlled commit scope.

The intended flow is:

1. create a short-lived feature branch from the default branch
2. implement one scoped change
3. run the verification scripts locally
4. commit only source code, configuration, and lightweight report files
5. push the branch and open a pull request
6. merge only after validation succeeds

This workflow was also reflected in practice during implementation through branch-based milestone cleanup and structural refactoring. The purpose of this versioning discipline is to keep changes auditable and reduce the risk of mixing runtime artifacts with source code changes.

Validation scripts used before merge include:

- ***scripts/run_train.py***
- ***scripts/run_compare.py***
- ***scripts/run_tests.py***
- ***scripts/run_zenml.py***

This satisfies the milestone requirement for code versioning and demonstrates that Milestone 4 is developed and maintained as a real repository component rather than as a one-off prototype.

4.3 Experiment Tracking and Model Versioning

Milestone 4 integrates MLflow for experiment tracking and local model versioning.

The MLflow integration is implemented in ***src/m4_model_dev/tracking/mlflow_utils.py.*** It provides the following capabilities:

- configuration of the local tracking URI
- creation of experiments when missing
- logging of run parameters
- logging of split metrics
- logging of supporting artifacts such as summaries and PNG figures
- logging of sklearn model artifacts
- registration of the selected model in a local model registry

The local MLflow store consists of:

- ***mlflow.db:*** tracking database
- ***mlruns/:*** artifact storage

Two main experiment types are logged:

- single-model training runs
- multi-model comparison runs

The currently supported model families are:

- dummy_prior
- logreg_numpy

- logreg_sklearn
- random_forest
- hist_gradient_boosting

Model selection is based on validation metrics only, which prevents leakage from the test split into selection decisions. The current selected model is random_forest. The local registered model name is: ***m4-facility-opening-best***

The latest local results after refactoring and verification are:

- **validation F1:** 0.6349
- **validation ROC-AUC:** 0.8944
- **test F1:** 0.3571
- **test ROC-AUC:** 0.7849

Milestone 4 also uses MLflow as a visualization layer. The local UI can be launched and inspected through the MLflow web interface at: ***http://127.0.0.1:5004***

Within the MLflow UI, the following are visible:

- experiment names
- run parameters
- metrics per split
- artifact files
- registered model versions
- attached training and comparison figures

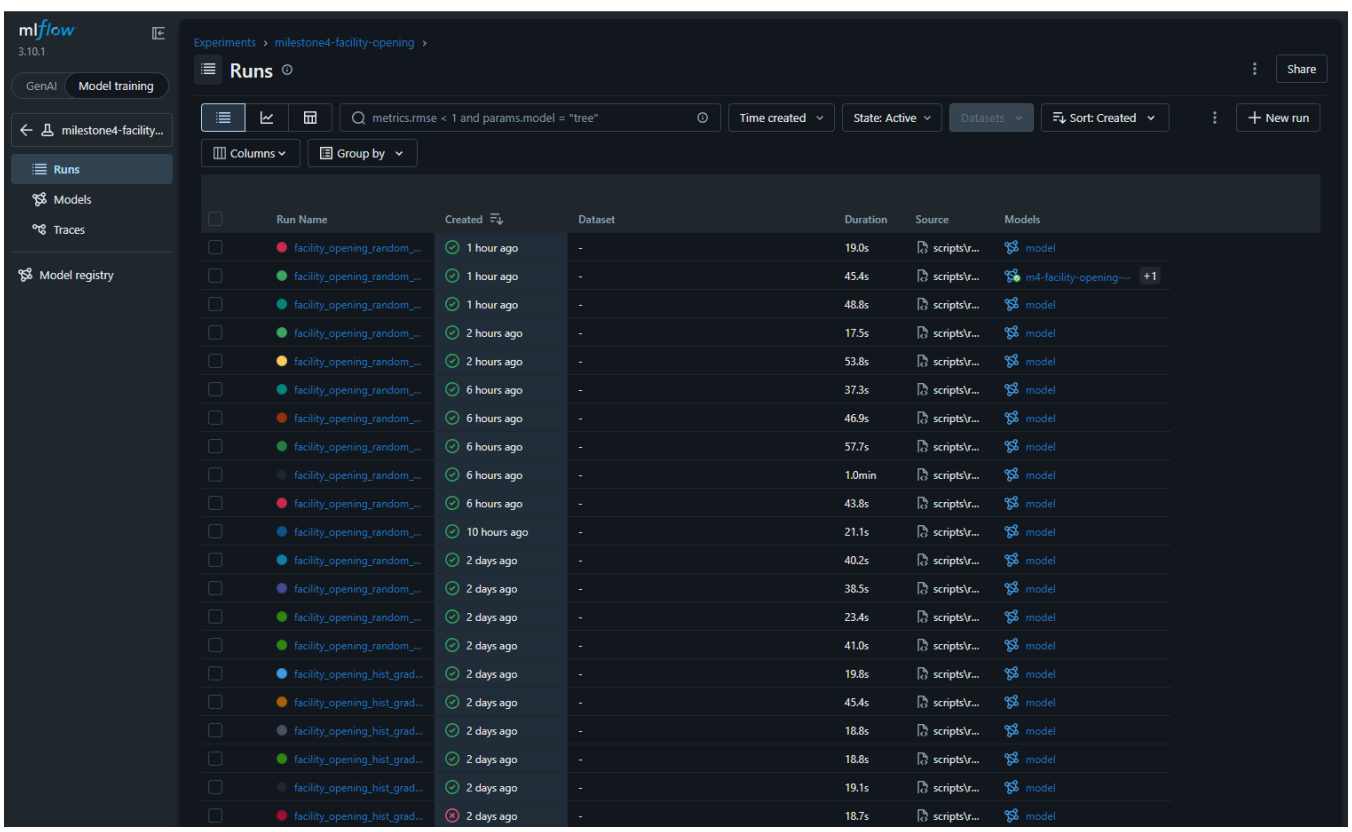


Figure 3: MLflow experiment list used for Milestone 4 experiment tracking

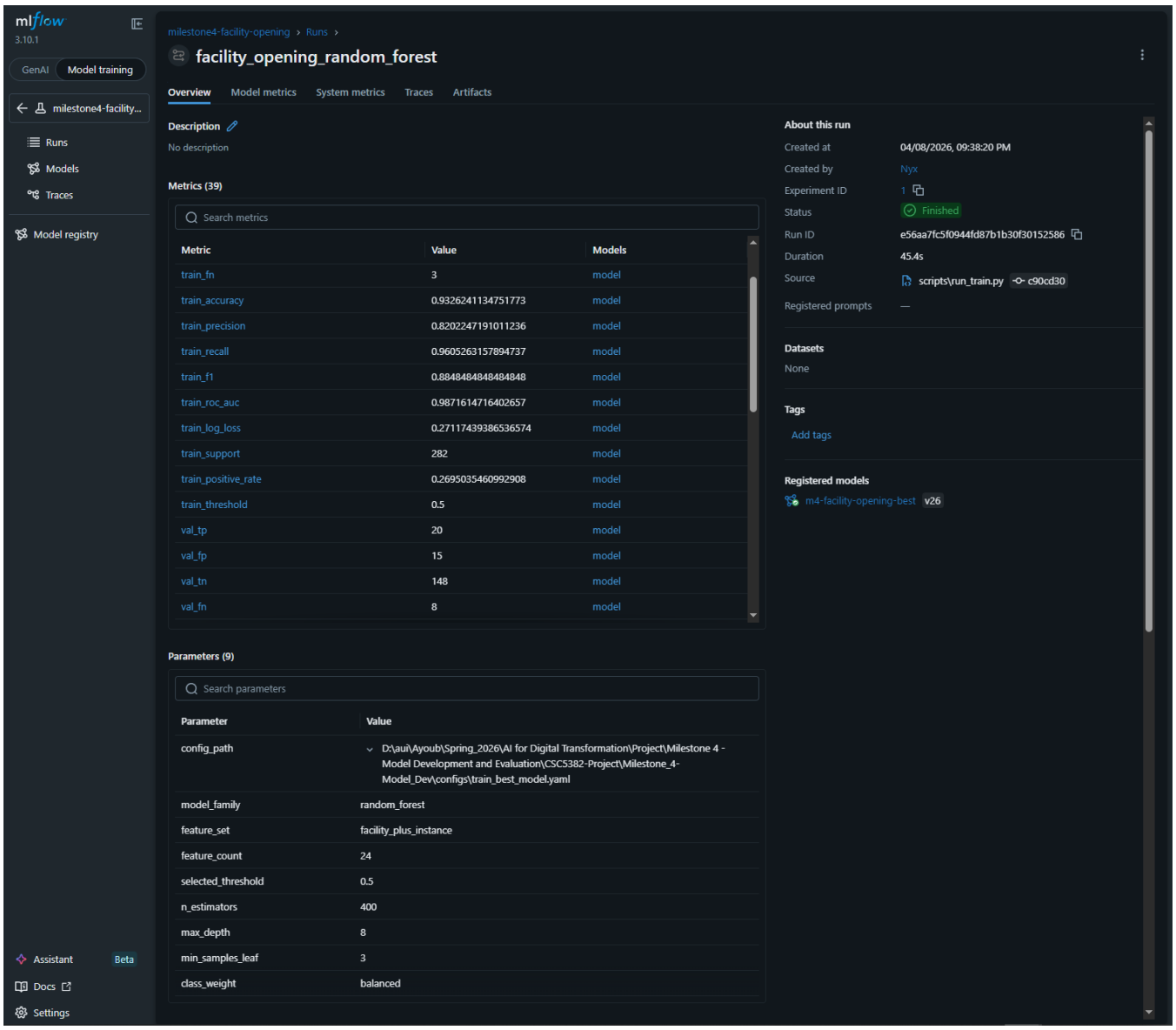


Figure 4: Example MLflow run for the selected random_forest model.

4.4 Integration of Model Training and Offline Evaluation into the ML Pipeline / MLOps Platform

Milestone 4 integrates the training and evaluation workflow into a local MLOps pipeline using **ZenML**.

The ZenML integration is implemented in ***src/m4_model_dev/pipelines/zenml_pipeline.py***. Rather than wrapping the entire workflow in a single opaque step, the pipeline is split into visible stages:

- load_config_step
- prepare_data_step
- train_model_step
- evaluate_model_step
- persist_outputs_step

This decomposition makes the pipeline easier to inspect and debug in the ZenML dashboard and aligns better with standard MLOps practice.

The ZenML pipeline performs:

1. configuration loading
2. label generation and dataset preparation
3. split generation
4. model fitting
5. offline evaluation
6. report writing
7. MLflow logging
8. artifact persistence

ZenML status is also written locally in:

- ***reports/zenml_status.txt***
- ***reports/zenml_status.json***

The local ZenML dashboard is available at: ***http://127.0.0.1:8238***

This UI provides visual access to pipeline runs and step execution. It complements MLflow by focusing on orchestration rather than experiment history.

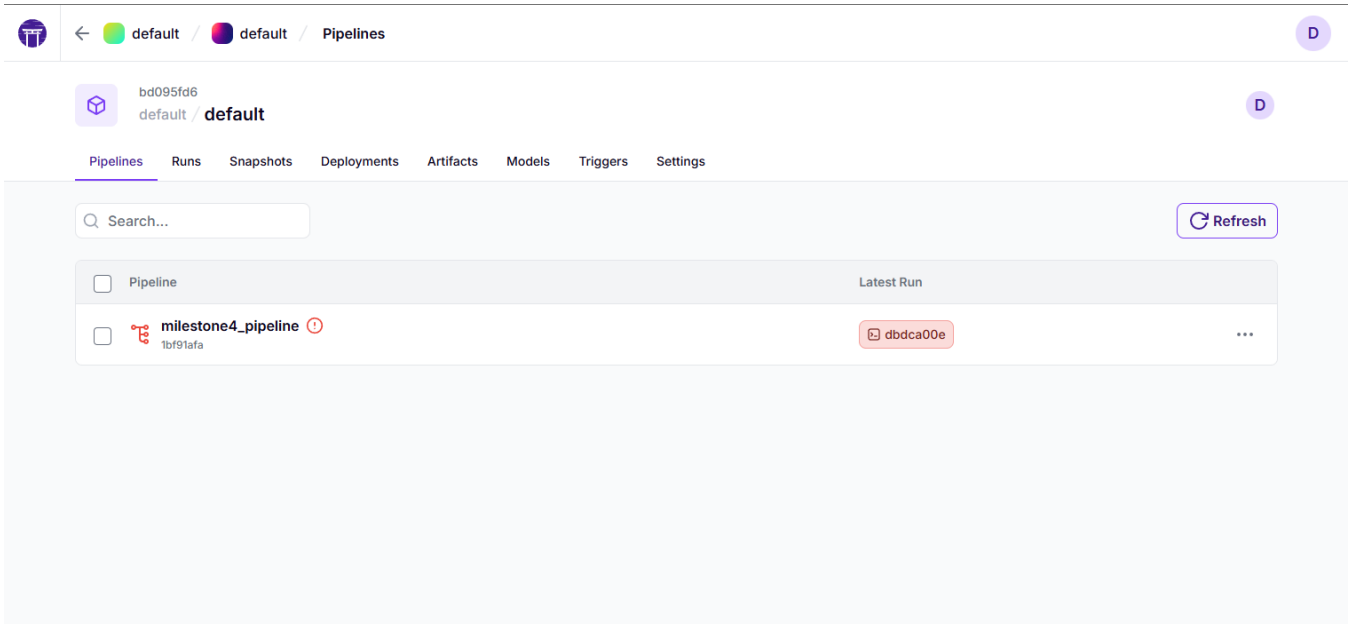


Figure 5: ZenML local dashboard used to visualize Milestone 4 pipeline runs.

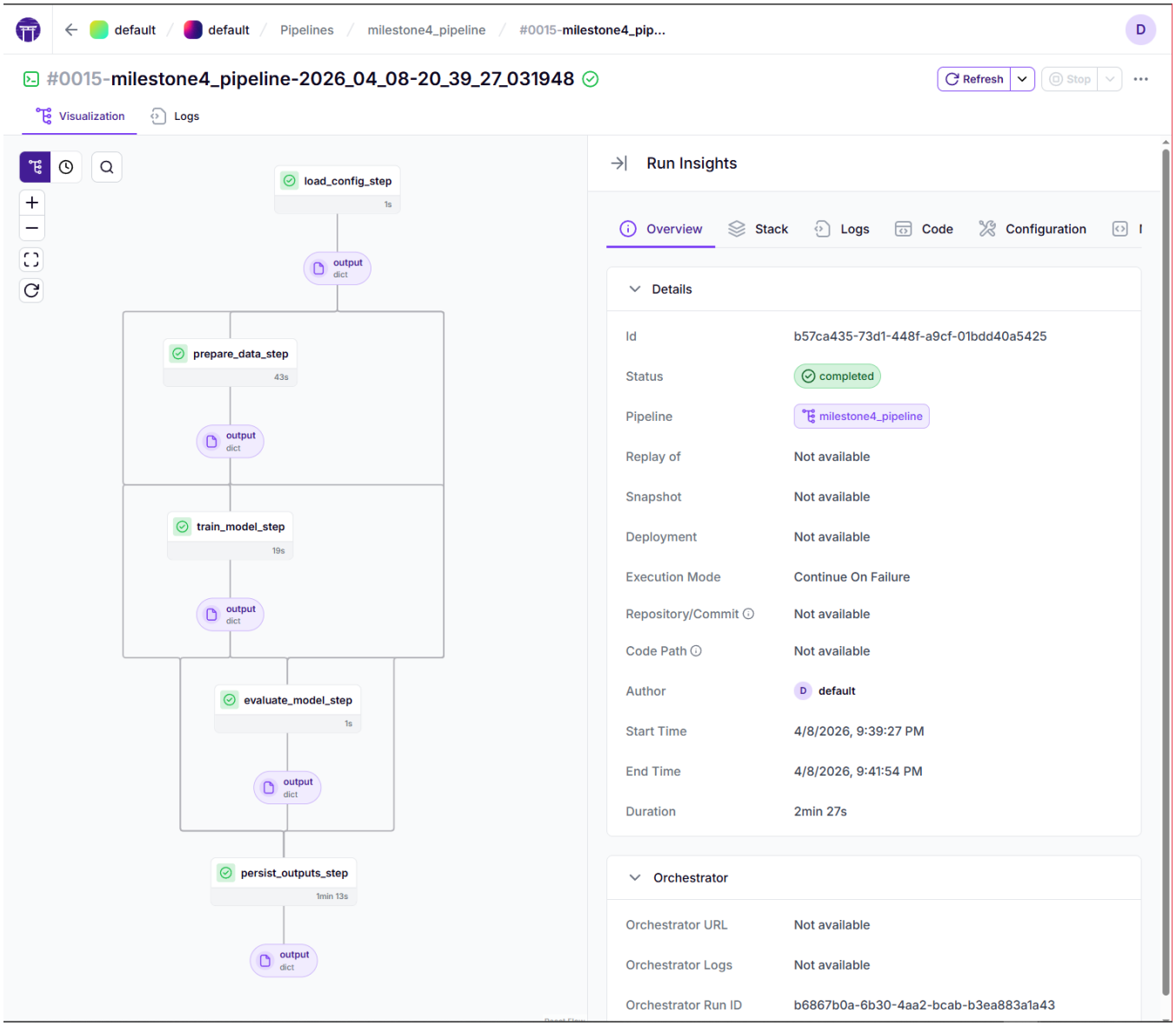


Figure 6: Multi-step ZenML pipeline for model training and offline evaluation.

The logical execution flow of the project is therefore:

- ***run_train.py*** -> ***training_pipeline.py*** -> data preparation -> model training -> evaluation -> reports -> MLflow
- ***run_compare.py*** -> ***comparison_pipeline.py*** -> shared training inputs -> train all candidate models -> rank on validation -> reports -> MLflow
- ***run_zenml.py*** -> ***zenml_pipeline.py*** -> same workflow orchestrated as explicit ZenML steps

This satisfies the milestone requirement of integrating model training and offline evaluation into an MLOps platform.

4.5 Energy Efficiency Measurement

As an optional bonus feature, Milestone 4 integrates CodeCarbon for energy-efficiency measurement during model training.

The integration is activated from the training workflow through ***src/m4_model_dev/tracking/codecarbon_utils.py.*** When enabled in configuration, the pipeline starts a CodeCarbon tracker before model fitting and stops it after training.

The output is written to: ***reports/emissions.csv***. The report contains estimated energy consumption statistics for CPU, RAM, and GPU activity during training.

This milestone also benefits from the CodeCarbon console output during execution, which helps validate that tracking is active. On the local Windows environment, CodeCarbon may fall back to CPU estimation mode when direct power measurement tools are unavailable. This is an implementation limitation of the environment rather than of the pipeline itself.

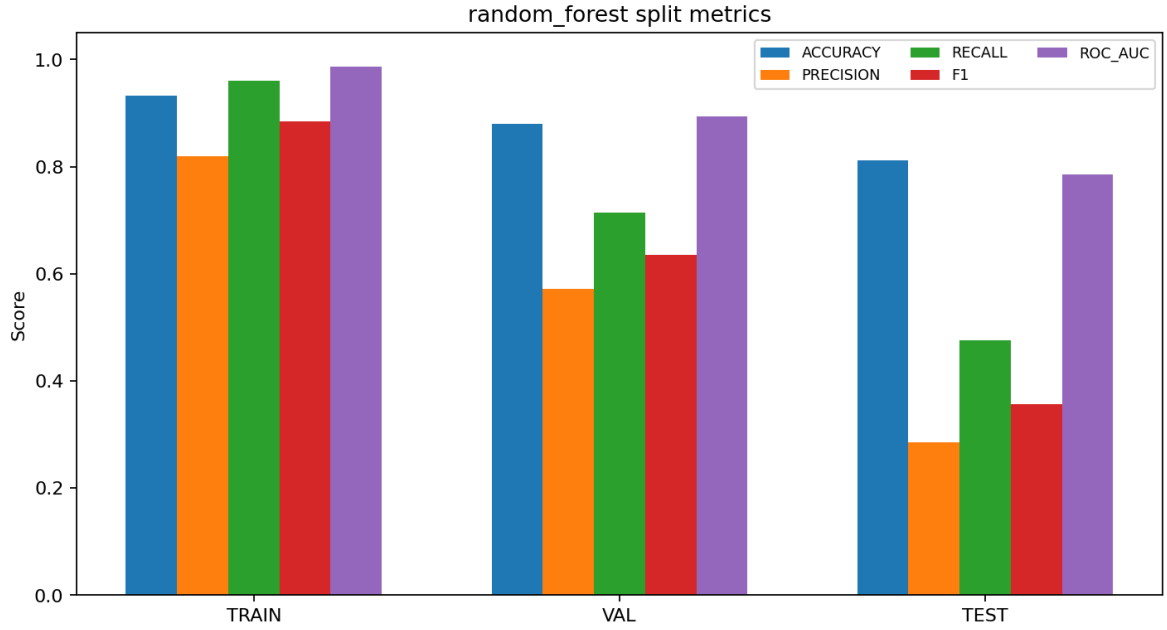
```
x64__qbz5n2kfra8p0\python3.13.exe" .\scripts\run_train.py
>>
[codecarbon WARNING @ 23:39:49] Multiple instances of codecarbon are allowed to run at the same time.
[codecarbon INFO @ 23:39:51] [setup] RAM Tracking...
[codecarbon INFO @ 23:39:51] [setup] CPU Tracking...
[codecarbon WARNING @ 23:39:55] No CPU tracking mode found. Falling back on estimation based on TDP for CPU.
 Windows OS detected: Please install Intel Power Gadget to measure CPU

[codecarbon INFO @ 23:39:55] CPU Model on constant consumption mode: Intel(R) Core(TM) i7-8750H CPU @ 2.20GHz
[codecarbon WARNING @ 23:39:55] No CPU tracking mode found. Falling back on CPU load mode.
[codecarbon INFO @ 23:39:55] [setup] GPU Tracking...
[codecarbon INFO @ 23:39:55] Tracking Nvidia GPUs via PyNVML
[codecarbon INFO @ 23:39:55] The below tracking methods have been set up:
                RAM Tracking Method: RAM power estimation model
                CPU Tracking Method: cpu_load
                GPU Tracking Method: pynvml

[codecarbon INFO @ 23:39:55] >>> Tracker's metadata:
[codecarbon INFO @ 23:39:55]   Platform system: Windows-10-10.0.19045-SP0
[codecarbon INFO @ 23:39:55]   Python version: 3.13.12
[codecarbon INFO @ 23:39:55]   CodeCarbon version: 3.2.6
[codecarbon INFO @ 23:39:55]   Available RAM : 15.859 GB
[codecarbon INFO @ 23:39:55]   CPU count: 12 thread(s) in 1 physical CPU(s)
[codecarbon INFO @ 23:39:55]   CPU model: Intel(R) Core(TM) i7-8750H CPU @ 2.20GHz
[codecarbon INFO @ 23:39:55]   GPU count: 1
[codecarbon INFO @ 23:39:55]   GPU model: 1 x NVIDIA GeForce GTX 1070
[codecarbon INFO @ 23:39:59] Emissions data (if any) will be saved to file D:\aui\Ayoub\Spring_2026\AI for Digital Transformation\Project\Milestone 4 - Model Development and Evaluation\CSC5382-Project\Milestone_4-Model_Dev\reports\
emissions.csv
[codecarbon INFO @ 23:40:01] Energy consumed for RAM : 0.000005 kWh. RAM Power : 10.0 W
[codecarbon INFO @ 23:40:01] Delta energy consumed for CPU with cpu_load : 0.000002 kWh, power : 4.931244 W
[codecarbon INFO @ 23:40:01] Energy consumed for All CPU : 0.000002 kWh
[codecarbon INFO @ 23:40:01] Monitoring GPUs with indices: [0] out of 1 total GPUs
[codecarbon INFO @ 23:40:01] Energy consumed for all GPUs : 0.000010 kWh. Total GPU Power : 15.95751666418003 2 W
[codecarbon INFO @ 23:40:01] 0.000017 kWh of electricity and 0.000000 L of water were used since the beginning.
2026/04/08 23:40:21 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitra
ry code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://scikit-learn.org/stable/model_persistence.html
Registered model 'm4-facility-opening-best' already exists. Creating a new version of this model...
Created version '29' of model 'm4-facility-opening-best'.
Milestone 4 training summary
run_name: facility_opening_random_forest
model_family: random_forest
feature_count: 24
selected_threshold: 0.50
model_artifact_path: D:\aui\Ayoub\Spring_2026\AI for Digital Transformation\Project\Milestone 4 - Model Development and Evaluation\CSC5382-Project\Milestone_4-Model_Dev\artifacts\random_forest_model.joblib
mlflow_logged: True
registered_model_name: m4-facility-opening-best
codecarbon_emissions_logged: True
mlflow_artifact_copy_warnings: 0
split_counts:
  - train: 282
  - val: 191
  - test: 191
metrics:
  - train: accuracy=0.9326, precision=0.8202, recall=0.9605, f1=0.8848, roc_auc=0.9872, log_loss=0.2712, tp=73, fp=16, tn=190, fn=3
  - val: accuracy=0.8796, precision=0.5714, recall=0.7143, f1=0.6349, roc_auc=0.8944, log_loss=0.4335, tp=20, fp=15, tn=148, fn=8
  - test: accuracy=0.8115, precision=0.2857, recall=0.4762, f1=0.3571, roc_auc=0.7849, log_loss=0.4941, tp=10, fp=25, tn=145, fn=11
```

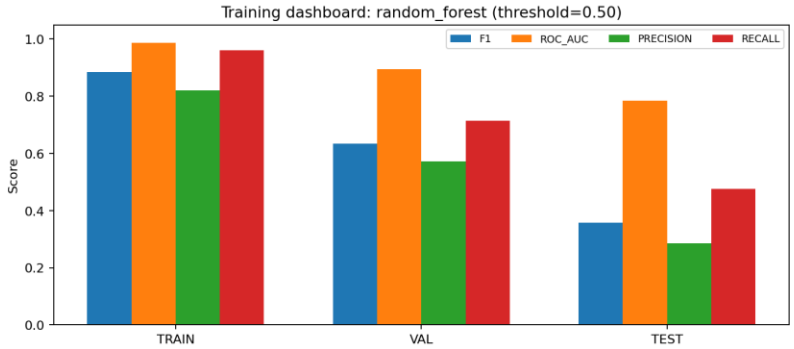
Figure 7: CodeCarbon execution log showing energy monitoring during the training phase.

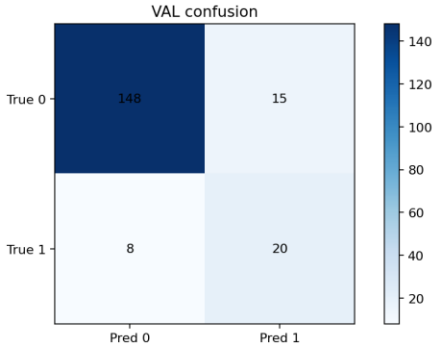
```
emissions.csv
CSC5382-Project > Milestone_4-Model_Dev > reports > emissions.csv > data
1  timestamp,project_name,run_id,experiment_id,duration,emissions,emissions_rate,cpu_power,gpu_power,ram_power,cpu_energy,gpu_energy,ram_energy,energy_consumed,water_consumed,country_name,country_iso_code,region,cloud_provider,c
2  2026-04-06T20:44:11,m4-facility-opening,48d508b2-44aa-4ea1-891d-ea0c8cee2de8,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,2.3981342000042787,7.4027202198998 53e-06,3.086866539781904 5e-06,5.211828000000001,5.8024978067588 85,10.0,2.7199
3  2026-04-06T20:46:43,m4-facility-opening,fe6cd3cf-f813-40d9-9055-3ac0a5ccef08,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,4.0991030000004685,2.0736899898076388e-05,5.058887248764917e-06,14.373965299500002,7.691263859696965,10.0,1.428
4  2026-04-06T20:48:03,m4-facility-opening,8da24a81-4f2d-43bb-b90a-7846f3c96b16,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,3.829988700003014,1.6364251410416704e-05,4.272663104829480 5e-06,9.633201160500002,7.444179107992402,10.0,8.8787
5  2026-04-06T20:50:12,m4-facility-opening,7b6d7faa-74ae-4895-846b-00216d33099d,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,3.810715300001904,1.4955805085441 05e-05,3.924671330191889e-06,7.525062396,7.264352074171028,10.0,6.908104478256
6  2026-04-06T20:51:15,m4-facility-opening,157f2324-869c-44ec-84b6-1e48a1d8c11b,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,3.816900000005262,1.4620860708416298e-05,3.830559016059143e-06,6.9817862295,7.297424967956455,10.0,6.3864485419
7  2026-04-06T20:52:58,m4-facility-opening,3c7ccada-a54d-4fed-94de-d60817b16e79,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,3.8464405000122497,1.559747219393957e-05,4.055040548239125e-06,8.296557142500001,7.360762838255513,10.0,7.6810
8  2026-04-06T20:56:28,m4-facility-opening,2527f254-ef18-43c8-b861-0e5742260c40,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,3.927081899993937,1.6971423878805032e-05,4.321637366114324 5e-06,9.893323359,7.377689511312509,10.0,9.4064907794
9  2026-04-06T20:58:55,m4-facility-opening,c7b1e713-29ec-4fa4-9b0b-c11ad44d9eff,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,3.8236378000001423,1.4417845609358708e-05,3.7707142683227 4e-06,6.7082730975,7.236658979032259,10.0,6.1336742882
10 2026-04-06T21:17:52,m4-facility-opening,d8eb6d94-32c1-49d9-b117-3e3fa23bc504,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,4.074848500007647,2.1462503928437528e-05,5.267067948267831e-06,14.187677355000002,8.937354707971597,10.0,1.4071
11 2026-04-06T21:20:17,m4-facility-opening,1e94e49d-50f9-4a05-9750-a2d0da515402,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,3.822063200001139,2.7902644608833 5e-05,7.300414238800403e-06,8.9537922225,25.31348198942108,10.0,8.2495889000
12 2026-04-06T21:30:59,m4-facility-opening,1cdfdf3a-dbd2-4dfe-b601-b0b1be16947d,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,2.4437443999922834,7.827343241099019e-06,3.203012246748774 4e-06,6.7048175295,5.086422794434997,10.0,3.6127024450
13 2026-04-06T21:32:32,m4-facility-opening,28dbf5ca-2661-4725-bd43-f9a273fc08be,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,4.42249090000405,3.0229872128535125e-05,6.835485433900484e-06,12.152529391500002,19.48531467265538,10.0,1.32223
14 2026-04-06T21:34:27,m4-facility-opening,c85f49cd-fc8a-418c-a233-d9e0a33f5cc6,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,4.651135799998883,1.9973344807384 53e-05,4.294294053377098e-06,9.5639112832 5,7.252909054319129,10.0,1.0960367724
15 2026-04-06T21:36:10,m4-facility-opening,b5a9ff37-aa50-4e8f-9b45-b4774bf4ff9e,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,4.195779699992272,1.6171026229963674e-05,3.854117085793008e-06,9.173706156,5.309921359039005,10.0,9.3477494543
16 2026-04-08T13:25:14,m4-facility-opening,79cc6fe0-ed22-4859-add8-926c4a817ce1,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,2.367285100015579,7.394959431896939e-06,3.123814462334204 3e-06,5.379564379500000 5,5.768216064303469,10.0,2.7850
17 2026-04-08T16:46:41,m4-facility-opening,1d6d3b86-fb6d-4421-a9e2-c8d5e3f2f381,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,2.712041100021452,8.793885626107028e-06,3.242534055268362 3e-06,6.926201687250001,4.894079023236322,10.0,4.21732
18 2026-04-08T16:47:31,m4-facility-opening,70c36a26-d11c-4240-ae0d-19ac30e77026,5b0fa12a-3dd7-45bb-9766-cc326314d9f1,4.450591100001475,1.9921333677325328e-05,4.476109628970122e-06,10.999877724,7.003300786896166,10.0,1.2054925801
```

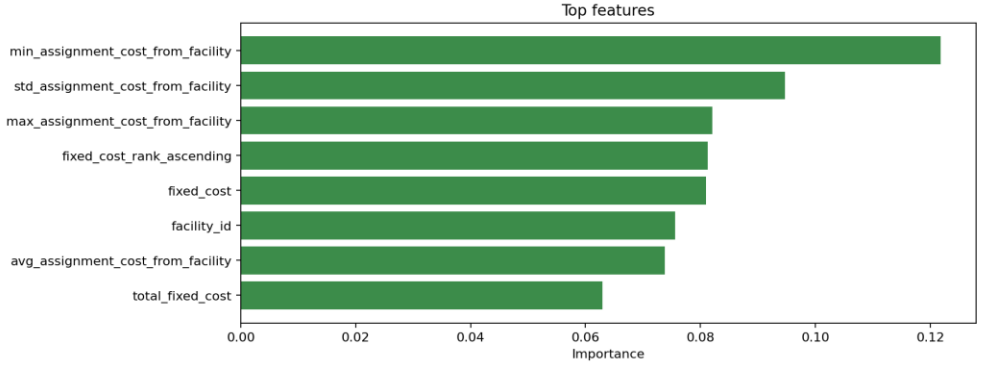
Figure 8: Example of the energy-efficiency report produced by CodeCarbon for a training run

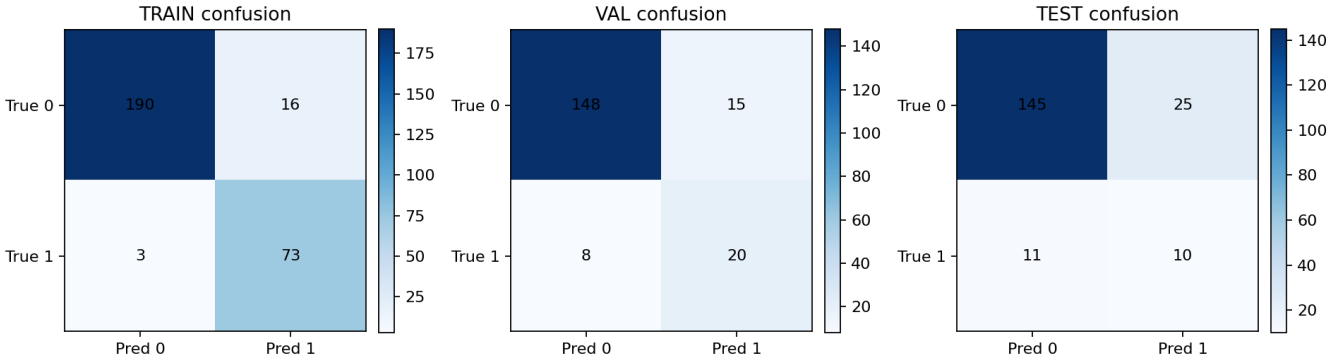
5. Visual Outputs:

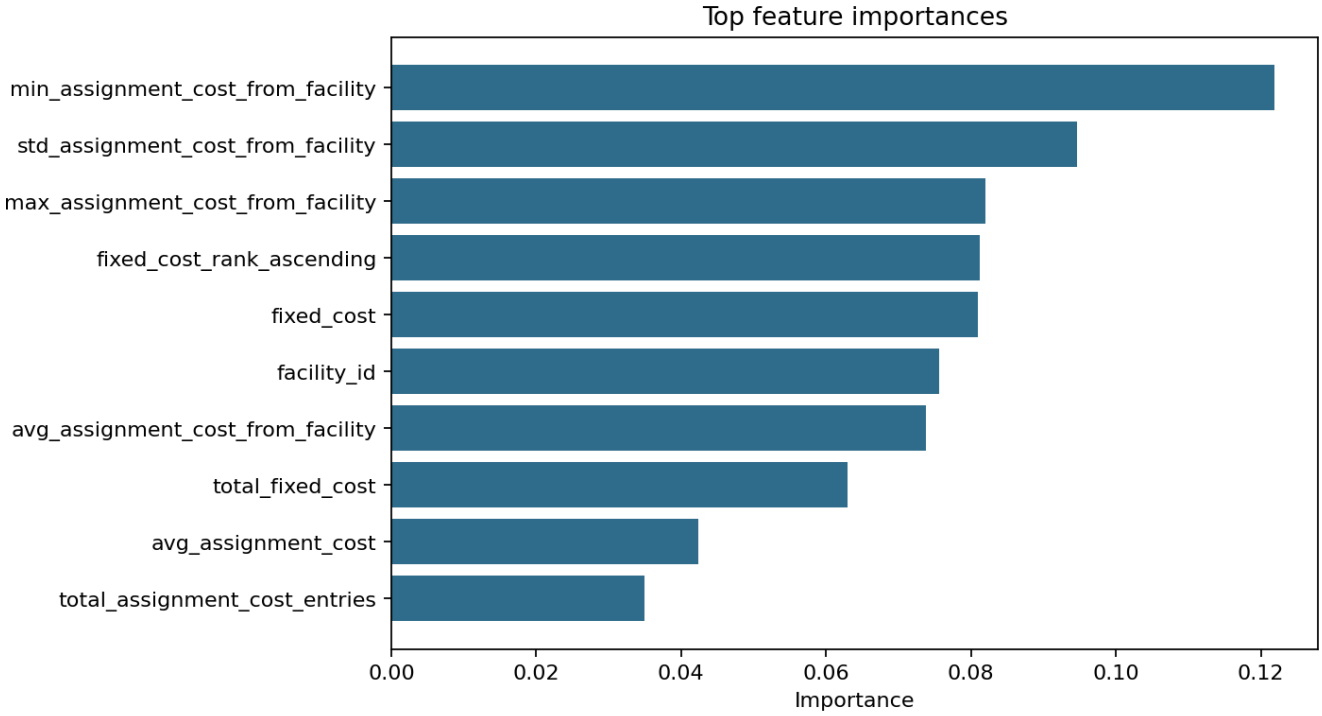


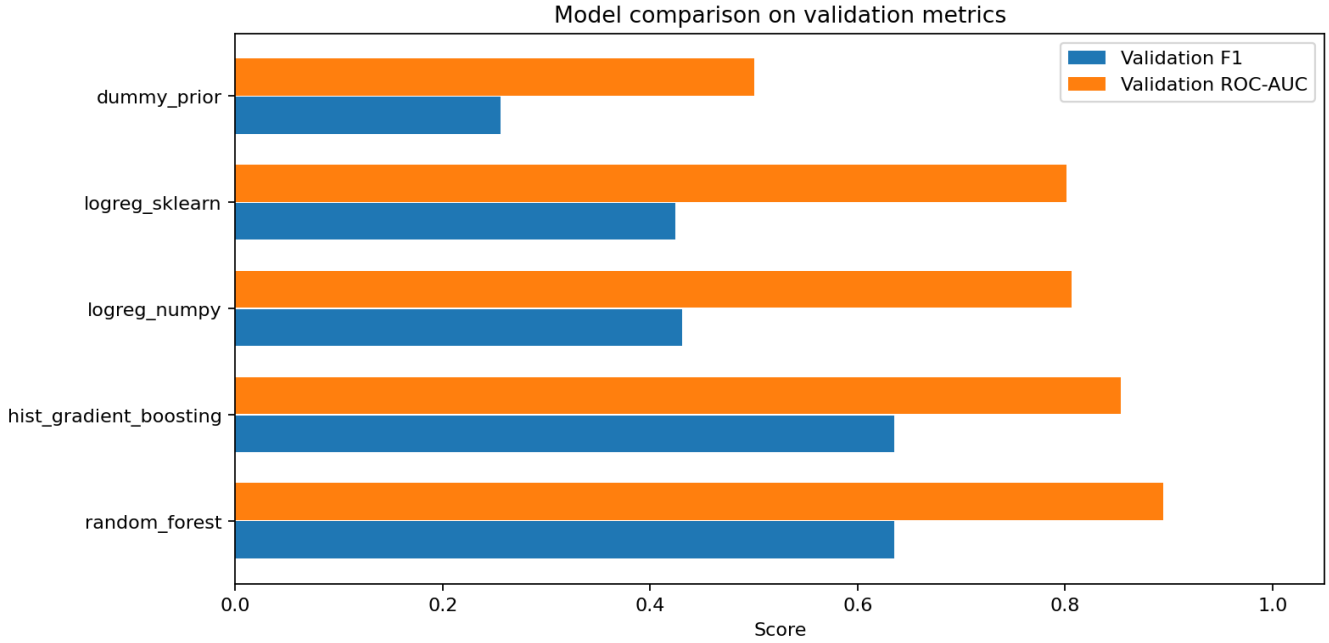


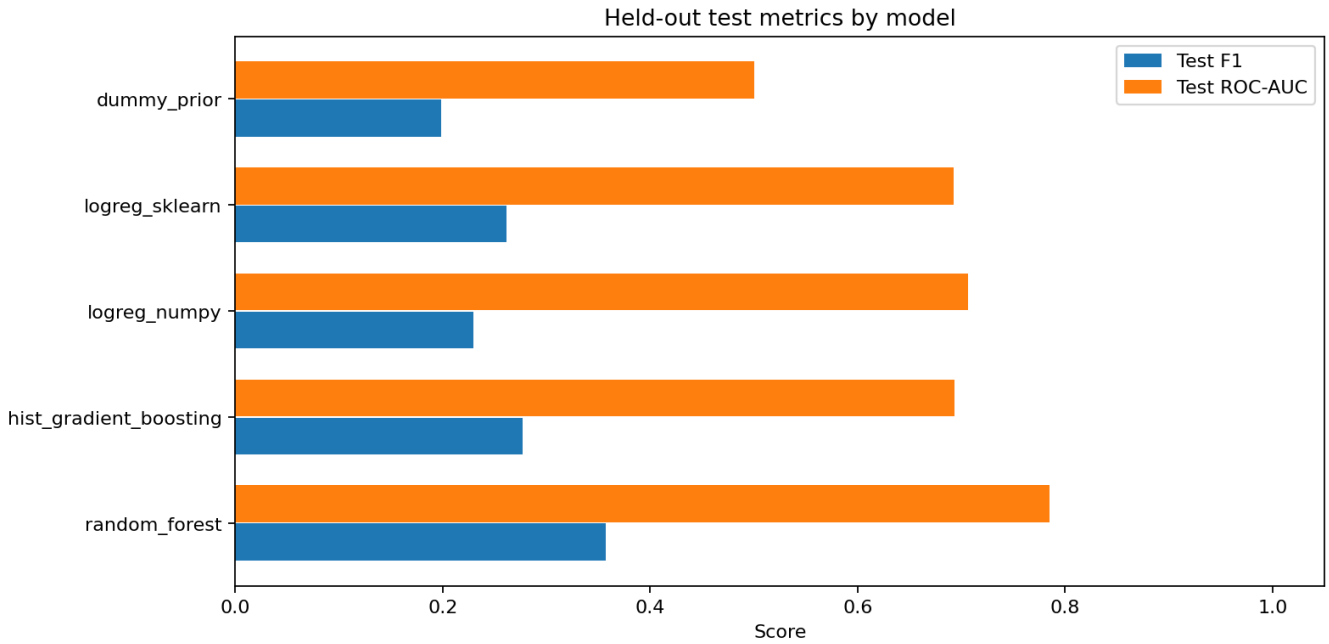


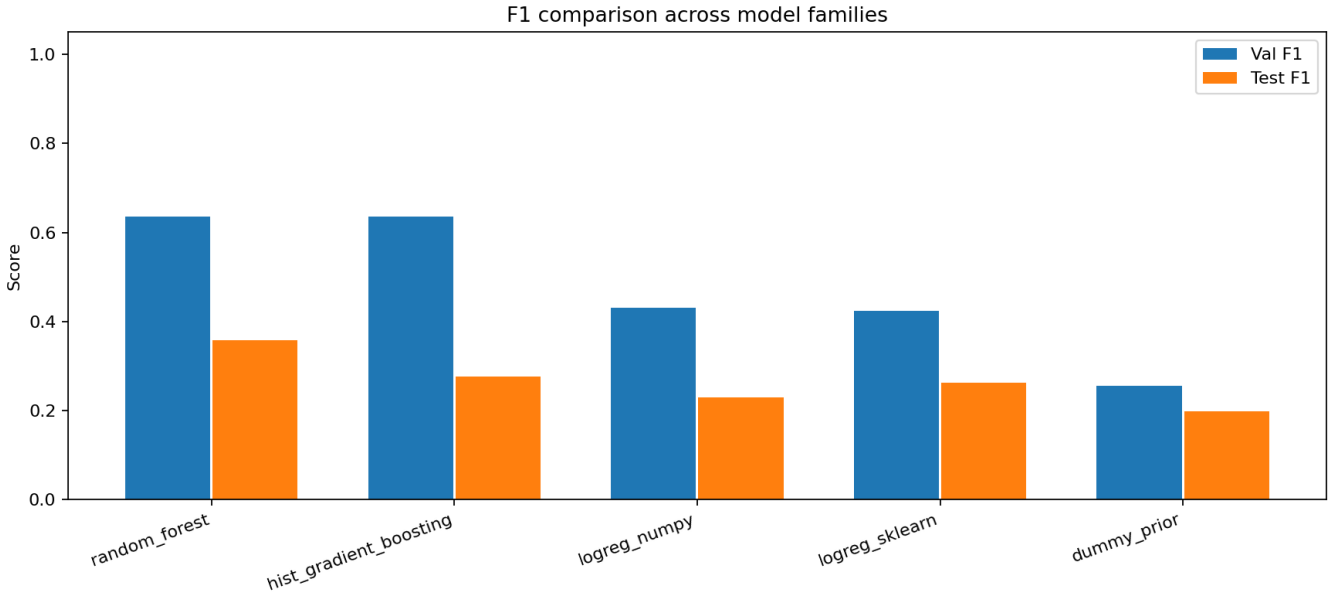


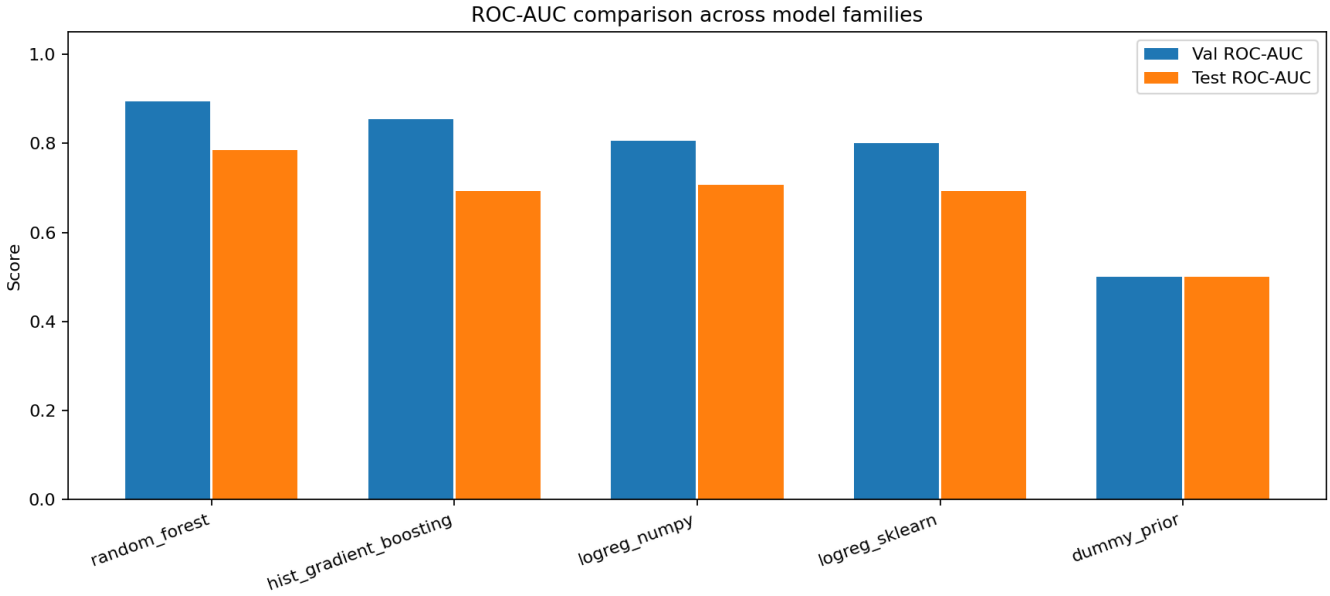

TRAIN confusion
True 0
True 1
190
16
3
73
Pred 0
Pred 1
VAL confusion
148
15
8
20
TEST confusion
145
25
11
10


Top feature importances
min_assignment_cost_from_facility
std_assignment_cost_from_facility
max_assignment_cost_from_facility
fixed_cost_rank_ascending
fixed_cost
facility_id
avg_assignment_cost_from_facility
total_fixed_cost
avg_assignment_cost
total_assignment_cost_entries
0.00
0.02
0.04
0.06
0.08
0.10
0.12
Importance


Model comparison on validation metrics
Validation F1
Validation ROC-AUC
dummy_prior
logreg_sklearn
logreg_numpy
hist_gradient_boosting
random_forest
0.0
0.2
0.4
0.6
0.8
1.0
Score


Held-out test metrics by model
Test F1
Test ROC-AUC
dummy_prior
logreg_sklearn
logreg_numpy
hist_gradient_boosting
random_forest
0.0
0.2
0.4
0.6
0.8
1.0
Score


F1 comparison across model families
Val F1
Test F1
Score
0.0
0.2
0.4
0.6
0.8
1.0
random_forest
hist_gradient_boosting
logreg_numpy
logreg_sklearn
dummy_prior


ROC-AUC comparison across model families
Val ROC-AUC
Test ROC-AUC
Score
0.0
0.2
0.4
0.6
0.8
1.0
random_forest
hist_gradient_boosting
logreg_numpy
logreg_sklearn
dummy_prior

6. Limitations and Next Steps

Current limitations:

- The predictive model is an auxiliary supervised layer and does not replace the symbolic solver.
- The benchmark remains relatively small, with only 15 UFLP instances.
- ZenML on Windows still produces environment-specific warnings, including the default pickle materializer warning for the custom scaler object.
- CodeCarbon CPU measurement may use fallback estimation on the local machine.
- The ML workflow currently focuses on tabular supervised models and does not include LLM fine-tuning.

Possible future work:

- add more model families or tuning strategies
- integrate richer hyperparameter search
- add custom ZenML materializers for cleaner artifact handling
- extend the ML layer toward solver warm-starting or candidate facility screening
- connect the ML layer more explicitly to the LLM-generated modeling pipeline from Milestone 2
- package the milestone outputs more tightly for cloud or CI execution